

हेलो! आपका स्वागत है **Unit 15: Error Handling** की complete study class में.

मैंने आपके workspace में, unit 15 error handling folder के अंदर ये complete study files write कर दी हैं:

- Study Guide File: chapter15_error_handling.md
- Code Examples File: error_handling_examples.js
- Practice Exercises File: error_handling_practice.js

चलिए अब Errors, exceptions, try-catch handlers, finally blocks aur custom exception classes structures ko simple Hinglish explanation ke sath samajhna shuru karte hain.

PART 1: ERROR CLASSIFICATION

Programming errors ko unke patterns aur characteristics ke broad types mein group kiya jata hai.

1. Syntax, Runtime & Logical Errors

What is it?

- **Syntax Error:** JS grammar validation rule break (blocks compilation phase).
- **Runtime Error:** Executing calculations issues (crash programs e.g. undefined references).
- **Logical Error:** Calculates outputs incorrectly due to bad arithmetic rules or algorithms.

Simple examples

javascript

// Syntax Error

// let x = ; // grammar rule broken

// Runtime Error

// console.log(y); // y is not defined, runs into crash

// Logical Error

let itemPrice = 100;

let tax = 5;

let total = itemPrice - tax; // Logical error (should add tax, not subtract)

```
console.log(total); // 95 (incorrect output calculation)
```

Output of Logical Error 95

PART 2: EXCEPTIONS MONITORING & RESOLVING

Code crashes ko monitor karne ke mechanisms.

1. try-catch Blocks & Error Object (e)

What is it?

- **try:** Monitoring container wrapping code block targets.
- **catch:** Executes only when runtime exception triggers in try block, handles error safely.

Important properties of Error Object:

- **err.name:** Type label (e.g. ReferenceError).
- **err.message:** Description message.

Simple example

javascript

```
try {  
    let result = 100 / xVar;  
} catch (err) {  
    console.log("Error Name: " + err.name);  
    console.log("Error Message: " + err.message);  
}
```

Output Error Name: ReferenceError Error Message: xVar is not defined

2. finally Block

What is it? Block coordinates executing **unconditionally** after try/catch cycles finish.

Simple example

javascript

```
try {
```

```
    console.log("try runs");  
  } finally {  
    console.log("finally runs always!");  
  }
```

Output try runs finally runs always!

PART 3: THROW & CUSTOM EXCEPTIONS

1. throw Statement

What is it? Keyword used to trigger a manual custom exception.

Simple example

javascript

```
function checkLevel(level) {  
  if (level < 1) {  
    throw new Error("Level cannot be below 1!");  
  }  
  return "Valid level";  
}  
  
try {  
  checkLevel(0);  
} catch (err) {  
  console.log(err.message);  
}
```

Output Level cannot be below 1!

2. Custom Errors

What is it? Extending base Error template properties to assign custom Exception labels.

Simple example

javascript

```
class AuthError extends Error {  
    constructor(msg) {  
        super(msg);  
        this.name = "AuthError";  
    }  
}  
  
try {  
    throw new AuthError("Access Denied");  
} catch (err) {  
    console.log(err.name + ": " + err.message);  
}
```

Output AuthError: Access Denied

QUICK REVISION SUMMARY

- **Error categories:** Syntax (bad spelling/grammar code fails to parse), Runtime (active execution crashes), Logical (incorrect output results).
 - **try-catch:** Monitors execution runtime exceptions, prevents app crash by execution catch blocks.
 - **finally:** runs always to release database variables connections or clean resources.
 - **throw:** initiates custom alerts validation exceptions.
 - **Custom class:** extends Error sets custom class name properties.
-

IMPORTANT DEFINITIONS (INTERVIEW-READY)

1. **Logical Bug:** System outputs mismatch caused by code algorithm design errors without causing technical crashes.
2. **Exception Interceptor (catch):** Logical recovery container handling execution errors safely.
3. **Unconditional Finalizer (finally):** Code block guaranteed to execute after try/catch blocks exit.

4. **Validation Throw (throw):** Operational keyword manually signaling exceptions constraints.
 5. **ReferenceError:** Runtime crash triggered when lookup paths fail to find variable names in the scope chain.
-

IMPORTANT INTERVIEW QUESTIONS & ANSWERS

Q1. Difference between Syntax Error and Runtime Error?

Ans: Syntax Error violates language grammar rules and blocks the program from compiling. Runtime Error occurs during program execution due to invalid actions (like accessing an undefined variable), causing the application to crash.

Q2. What is a Logical Error?

Ans: Logical Error happens when the code builds and executes without crashes, but yields incorrect outputs due to bad algorithms or formulas.

Q3. Explain the flow of try-catch-finally.

Ans: The program executes the code in the try block. If an error occurs, execution jumps to the catch block. The finally block runs immediately after, regardless of whether an error occurred.

Q4. Predict output:

javascript

```
try {  
    console.log("A");  
    throw new Error("B");  
    console.log("C");  
} catch(err) {  
    console.log(err.message);  
}
```

Ans: "A" "B" (Execution inside try halts and jumps to catch immediately after throw).

Q5. What properties does the Error object have?

Ans: The standard properties are name (the type of error) and message (description of the error).

Q6. Difference between throw and try-catch?

Ans: obj.throw is used to trigger an error manually. try-catch is used to catch and handle errors so that the program does not crash.

Q7. Why do we need the finally block?

Ans: To run cleanup code (like closing database connections or clearing resources) that must run whether the program succeeded or failed.

Q8. Predict output:

javascript

```
try {  
    console.log(10 / 0);  
} catch (err) {  
    console.log("Error");  
}
```

Ans: Infinity. Division by zero in JavaScript returns Infinity and does not throw a runtime error, so the catch block is not executed.

Q9. How do you create a custom error class?

Ans: By extending the built-in Error class and calling super(message) in the constructor.

Q10. Predict output:

javascript

```
try {  
    let x = 10;  
} finally {  
    console.log("Finally");  
}
```

Ans: "Finally". (finally runs even if there is no catch block).

Q11. Predict output:

javascript

```
const name = "Ravi";  
name = "Amit"; // TypeError inside try checks
```

Ans: TypeError (Cannot assign to read-only property).

Q12. Predict output:

```
javascript
try {
    let x = y;
} catch (e) {
    console.log(e instanceof ReferenceError);
}
```

Ans: true. Accessing an undefined variable throws a ReferenceError.

Q13. Can you nest try-catch blocks?

Ans: Yes. A try-catch block can be placed inside another try block to handle nested operations.

Q14. What does throw "Error!" do?

Ans: It throws a string exception. JavaScript allows throwing any type of value, but throwing an instance of Error is recommended for stack trace support.

Q15. Predict output:

```
javascript
function check(x) {
    if (!x) throw new Error("Missing");
}
try {
    check("");
} catch(e) {
    console.log(e.message);
}
```

Ans: "Missing". (An empty string is falsy, triggering the throw statement).

Q16. Difference between e.target and error checking properties?

Ans: e.target is an event target reference. Error checking properties like err.message hold error metadata.

Q17. Predict output:

javascript

```
try {  
    console.log("try");  
} catch(err) {  
    console.log("catch");  
} finally {  
    console.log("finally");  
}
```

Ans: "try" "finally"

Q18. What error is thrown when parsing invalid JSON?

Ans: SyntaxError. (e.g. JSON.parse("{invalid}") throws SyntaxError).

Q19. Predict output:

javascript

```
try {  
    JSON.parse("abc");  
} catch(err) {  
    console.log(err.name);  
}
```

Ans: "SyntaxError".

Q20. Does finally execute if the catch block throws another error?

Ans: Yes. The finally block executes before the new error propagates up the call stack.



PRACTICE QUESTIONS

Question 1: Check runtime reference crash

- **Question:** Write a function checking if variables are defined using try-catch blocks.
- **Solution:**

javascript


```
function checkVar() {
  try {
    console.log(mySecretVar);
  } catch (err) {
    console.log("Captured reference exception: " + err.message);
  }
}
```

checkVar();

- **Output:** Captured reference exception: mySecretVar is not defined
- **Explanation:** Encapsulates the ReferenceError in try-catch to log a clean error message.

Question 2: Input validator throwing errors

- **Question:** Create input length check. If input length is less than 3, throw custom error "Input too short".
- **Solution:**

javascript

```
function validateName(name) {
  if (name.trim().length < 3) {
    throw new Error("Input too short");
  }
  return "Valid Name";
}

try {
  validateName("Ed");
} catch (err) {
  console.log("Failed validation: " + err.message);
}
```

- **Output:** Failed validation: Input too short

- **Explanation:** Checks input length, throws error using `throw new Error()`, which is caught and logged.

Question 3: finally execution verification

- **Question:** Verify finally executes even after function returns inside try block.
- **Solution:**

javascript

```
function testReturn() {  
  try {  
    return "Returned from try";  
  } finally {  
    console.log("Finally block executed!");  
  }  
}  
  
console.log(testReturn());
```

- **Output:** Finally block executed! Returned from try
- **Explanation:** JavaScript guarantees that the finally block runs before control returns to the caller.